

Newsbreak AI全栈工程师

一面（技术基础） · 1-6

1. 简单介绍一下 SSE 和 WebSocket 的区别，为什么 AI 对话场景多采用 SSE？
2. 在处理多个 AI 模型并行调用时，Promise.all 和 Promise.allSettled 有什么区别？
3. 如何使用 CSS 实现一个类似 ChatGPT 的对话气泡布局，要求头像固定，内容自适应？
4. 请手写一个简单的打字机效果函数，模拟 AI 逐字输出。
5. TypeScript 中的 Interface 和 Type 有什么区别？在定义 AI 接口返回数据时如何选择？
6. 算法题：给定一个字符串，找出其中不含有重复字符的最长子串的长度。

二面（深入原理与项目） · 1-8

1. 详细说明 RAG（检索增强生成）的前端实现流程，以及前端在其中扮演的角色。
2. 在处理 AI 流式输出时，Markdown 渲染经常会出现标签截断导致的样式错乱，你怎么解决？
3. 谈谈你对 Prompt Engineering 的理解，前端如何管理复杂的 Prompt 模板？
4. 什么是 AI Agent？在前端实现一个 Agent 循环（Plan-Action-Observe）的核心逻辑是什么？
5. 对话列表很长时，如何保证流畅度？如果对话中包含复杂的代码高亮和数学公式渲染呢？
6. Node.js 作为全栈中间层，在处理 LLM 长连接请求时有哪些需要注意的地方？
7. 如何实现一个前端本地的向量检索功能（例如在浏览器处理少量文档）？
8. 介绍一下你解决过的一个 AI 相关的技术难点。

三面（架构与综合能力） 1-5

1. 如果让你从零设计一个支持多模态（文字、图片、文件）的 AI 聊天系统，你会如何设计前端架构？
2. AI 产品的 Token 成本很高，作为前端，你有哪些手段可以帮助公司节省成本？
3. 如何建立 AI 生成内容的质量评估体系（Evaluation）？前端能做什么？
4. 谈谈 WebGPU 对未来前端 AI 领域的影响，你有哪些关注或尝试？
5. 在一个 AI 快速迭代的团队中，你如何平衡业务交付压力与技术架构的优雅性？

一面（技术基础）

1. 简单介绍一下 SSE 和 WebSocket 的区别，为什么 AI 对话场景多采用 SSE？

难度：★

考点：网络协议、流式传输

答案要点：

SSE 是基于 HTTP 的单向推送协议，而 WebSocket 是全双工的独立协议，AI 场景选择 SSE 主要是因为其轻量且原生支持断线重连。

- 协议层：SSE 运行在 HTTP 之上，开发成本低，兼容性好；WebSocket 是独立协议，需握手切换。
- 数据流向：SSE 仅支持服务端向客户端推送；WebSocket 支持双向实时通信。
- AI 适配：大模型生成是增量的，SSE 的 text/event-stream 格式天然契合流式输出。
- 维护成本：SSE 支持自动重连，且在代理服务器（如 Nginx）上的配置比 WebSocket 更简单。

常见坑：

- 误以为 SSE 只能传文本，其实可以通过 Base64 传二进制。
- 忽略 HTTP/1.1 下 SSE 有 6 个并发连接限制的问题。

追问：

- 如果要在 SSE 中实现客户端中断请求，前端该怎么做？

2. 在处理多个 AI 模型并行调用时，Promise.all 和 Promise.allSettled 有什么区别？

难度：★

考点：异步编程、并发控制

答案要点：

Promise.all 要求所有任务成功，而 Promise.allSettled 会等待所有任务结束并返回每个任务的状态结果。

- 失败处理：Promise.all 只要有一个 reject 就会立即触发 catch；allSettled 会记录每个 promise 的 status（fulfilled 或 rejected）。
- 返回值：all 返回结果数组；allSettled 返回包含 status 和 value/reason 的对象数组。
- 场景选择：AI 场景中如果调用多个模型做对比，通常用 allSettled 以确保部分模型失败时不影响其他结果展示。

常见坑：

- 在 Promise.all 中忘记处理 catch 导致整个链路崩溃。

追问：

- 如何实现一个带并发数限制的 Promise 调度器？

3. 如何使用 CSS 实现一个类似 ChatGPT 的对话气泡布局，要求头像固定，内容自适应？

难度：★

考点：CSS 布局、Flexbox

答案要点：

通常使用 Flexbox 布局，通过 flex-shrink 属性控制头像不被压缩，内容区域设置 flex-grow 自动填充。

- 容器属性：display: flex; align-items: flex-start;
- 头像处理：设置固定宽高，并设置 flex-shrink: 0 防止被内容挤压。
- 内容处理：使用 word-break: break-word 处理长文本，max-width 限制最大宽度。
- 左右对齐：通过 flex-direction: row-reverse 或 margin-left: auto 实现用户侧气泡靠右。

常见坑：

- 长字符串（如 URL）导致气泡溢出容器。
- 头像在内容过少时变形。

4. 请手写一个简单的打字机效果函数，模拟 AI 逐字输出。

难度：★★

考点：定时器、字符串操作

答案要点：

打字机效果核心是利用 setInterval 或递归的 setTimeout，将目标字符串按索引逐个提取并更新到 DOM 中。

- 核心逻辑：维护一个当前索引指针，每隔固定时间截取 substring(0, index++)。
- 性能考虑：对于超长文本，频繁操作 DOM 会有性能问题，可以考虑 requestAnimationFrame。
- 交互细节：需要提供停止回调和结束状态标识。

常见坑：

- 忘记在组件销毁时清除定时器，导致内存泄漏。
- 字符包含 Emoji 或特殊字符时，按索引截取可能导致乱码。

代码块

```
1  function typeWriter(text, callback, speed = 50) {
2    let index = 0;
3    const timer = setInterval(() => {
4      if (index < text.length) {
5        callback(text.substring(0, index + 1));
6        index++;
7      } else {
8        clearInterval(timer);
9      }
10   }, speed);
11   return () => clearInterval(timer);
12 }
```

5. TypeScript 中的 Interface 和 Type 有什么区别？在定义 AI 接口返回数据时如何选择？

难度：★

考点：TypeScript 基础

答案要点：

Interface 主要用于定义对象形状且支持声明合并，Type 更加灵活，支持联合类型、元组等定义。

- 扩展性：Interface 可以通过 extends 继承，也可以多次声明自动合并；Type 通过 & 交叉类型模拟继承。
- 能力范围：Type 可以定义基本类型别名、联合类型（如 'success' | 'error'），Interface 不行。
- 建议：定义 AI 响应结构等固定对象时首选 Interface，定义复杂的 Prompt 模板类型或状态联合时使用 Type。

常见坑：

- 混淆声明合并（Declaration Merging）的概念。

追问：

- 什么是泛型？如何写一个通用的 AI API 响应包装类型？

6. 算法题：给定一个字符串，找出其中不含有重复字符的最长子串的长度。

难度：★★

考点：滑动窗口、字符串处理

答案要点：

使用滑动窗口算法，维护一个左指针和右指针，利用哈希表记录字符最后出现的位置。

- 初始化：left 指针为 0，maxLen 为 0，Map 记录字符位置。
- 遍历：右指针向右移动，若遇到重复字符且在窗口内，则移动左指针到重复字符下一位。
- 更新：每次移动右指针后计算当前窗口长度并更新 maxLen。

常见坑：

- 左指针移动时没有判断重复字符是否在当前窗口内。
- 字符串为空或长度为 1 的边界情况。

二面（深入原理与项目）

1. 详细说明 RAG（检索增强生成）的前端实现流程，以及前端在其中扮演的角色。

难度：★★★★

考点：RAG 架构、前端工程化

答案要点：

RAG 流程包括文档切片、向量化、检索和生成，前端主要负责上传交互、检索结果展示以及引用来源的溯源。

- 上传阶段：前端处理文件切片上传，展示解析进度和状态。
- 检索阶段：前端调用向量数据库接口或后端 API，获取 Top-K 相关片段。
- 渲染阶段：将检索到的知识库片段与 AI 生成内容关联，实现点击引用标注跳转到原文位置。
- 优化：前端可以做本地 Embedding 缓存，减少重复检索。

常见坑：

- 引用标注的索引匹配错误。
- 大文件上传时的内存占用问题。

追问：

- 如何在前端实现 PDF 文件的内容高亮定位？

2. 在处理 AI 流式输出时，Markdown 渲染经常会出现标签截断导致的样式错乱，你怎么解决？

难度：★★

考点：流式渲染、DOM 更新优化

答案要点：

这是因为流式传输中 HTML 标签或 Markdown 语法（如代码块反引号）未闭合，通常通过增量解析器和缓冲区处理。

- 增量解析：使用支持流式的 Markdown 解析器（如 markdown-it 的插件或 custom parser），在解析前补齐未闭合的标签。
- 缓冲区策略：设置一个小型的 Buffer，当遇到可能产生截断的特殊符号（如 ```）时暂存，等下一段数据到达后再解析。
- 视觉优化：在未完成的代码块下方显示加载动画或占位符。

常见坑：

- 每次收到新字符都全量重新渲染 Markdown，导致 CPU 占用过高。

追问：

- 如何在 React 中优化这种高频更新的组件？

3. 谈谈你对 Prompt Engineering 的理解，前端如何管理复杂的 Prompt 模板？

难度：★★

考点：Prompt 优化、工程化管理

答案要点：

Prompt Engineering 是通过结构化指令引导模型输出，前端通常采用模板化、版本化和动态组装的方式进行管理。

- 结构化设计：采用 Role (角色)、Context (上下文)、Task (任务)、Constraint (约束) 的四段式设计。
- 模板管理：将 Prompt 抽离为独立配置文件或数据库存储，支持变量插值（如 `{{user_input}}`）。
- 动态组装：根据用户交互状态（如历史对话、当前页面上下文）动态拼接最终的 System Prompt。
- 版本控制：对 Prompt 进行 A/B 测试，记录不同版本 Prompt 的模型表现。

常见坑：

- 硬编码 Prompt 到代码中，导致修改需重新发布。
- 忽略了 Token 长度限制，未对 Context 进行截断处理。

4. 什么是 AI Agent？在前端实现一个 Agent 循环（Plan-Action-Observe）的核心逻辑是什么？

难度：★★★

考点：AI Agent、异步流控制

答案要点：

AI Agent 是能够自主规划并调用工具完成任务的系统，其核心是一个基于模型反馈的递归或循环逻辑。

- Plan（规划）：将用户目标分解为多个子任务或工具调用指令。
- Action（执行）：前端根据模型返回的 Tool Call 指令，执行对应的本地函数（如查询 API、操作 DOM）。
- Observe（观察）：将执行结果（成功/失败/数据）反馈给模型，由模型决定下一步动作。
- 终止条件：模型判断任务完成或达到最大迭代次数。

常见坑：

- 陷入无限循环（模型反复调用同一个错误工具）。
- 异步工具调用顺序错乱。

5. 对话列表很长时，如何保证流畅度？如果对话中包含复杂的代码高亮和数学公式渲染呢？

难度：★★

考点：性能优化、虚拟列表

答案要点：

核心方案是采用虚拟列表（Virtual List）技术，只渲染可视区域内的消息，并对重型组件进行按需加载。

- 虚拟滚动：使用 `react-window` 或 `react-virtuoso`，固定或动态计算消息高度。
- 延迟渲染：代码高亮（`Prism.js`）和公式渲染（`KaTeX`）只在消息进入视口时触发。
- Web Worker：将复杂的 Markdown 解析或语法高亮逻辑移出主线程。

- 缓存：对已渲染的消息进行 Memo 缓存，避免因新消息加入导致旧消息重绘。

常见坑：

- 动态高度计算不准确导致列表跳动。
- 搜索对话内容时，虚拟列表无法直接定位。

6. Node.js 作为全栈中间层，在处理 LLM 长连接请求时有哪些需要注意的地方？

难度：★★

考点：Node.js 后端开发、流处理

答案要点：

Node.js 需处理好流的转发、超时管理以及内存压力，确保高并发下的稳定性。

- 流转发：使用 `stream.pipe()` 或 `pipeline()` 将模型响应直接导向客户端，避免全量内存缓冲。
- 超时控制：设置合理的 Socket 超时，防止模型响应过慢导致连接堆积。
- 异常捕获：监听 stream 的 error 事件，防止因模型侧断开导致 Node 进程崩溃。
- 安全校验：在中间层进行 Token 鉴权和敏感词过滤（Moderation）。

常见坑：

- 忘记设置 `res.setHeader('Content-Type', 'text/event-stream')`。
- 未处理客户端主动断开连接的情况，导致后端继续消耗 Token。

7. 如何实现一个前端本地的向量检索功能（例如在浏览器处理少量文档）？

难度：★★★

考点：端侧 AI、向量计算

答案要点：

在浏览器端可以利用 Transformers.js 进行 Embedding 计算，并结合简单的余弦相似度算法进行检索。

- 模型加载：使用 Transformers.js 加载轻量级的 Embedding 模型（如 Xenova/all-MiniLM-L6-v2）。
- 向量存储：将计算出的向量存储在 IndexedDB 中。
- 相似度计算：实现余弦相似度函数，对用户查询向量与库中向量进行点积运算并排序。
- 性能：对于万级以上数据，建议使用 WebAssembly 优化的向量库。

常见坑：

- 模型文件过大导致首屏加载慢。
- 频繁计算导致主线程卡顿。

8. 介绍一下你解决过的一个 AI 相关的技术难点。

难度：★★

考点：问题解决能力、项目深度

答案要点：

- (此题需结合候选人实际经验，以下为示例)
- 解决过大模型输出不稳定性导致的 UI 崩溃问题，通过引入 Schema 校验和容错机制提升了健壮性。
- 问题背景：模型有时不按 JSON 格式返回，导致前端 JSON.parse 报错。
 - 解决方案：引入 Zod 进行 Schema 校验，并编写了一个修复函数，利用正则提取字符串中的 JSON 部分。
 - 兜底策略：当解析失败时，展示原始文本并引导用户重新生成。
 - 结果：系统稳定性提升，异常报错率降低了 80%。

三面（架构与综合能力）

1. 如果让你从零设计一个支持多模态（文字、图片、文件）的 AI 聊天系统，你会如何设计前端架构？

难度：★★★
考点：系统设计、架构思维
答案要点：

- 架构应采用分层设计，将消息协议层、状态管理层和渲染引擎解耦，以支持灵活的功能扩展。
- 协议层：定义统一的消息模型（Message Schema），兼容不同类型媒体的 metadata。
 - 状态层：使用 Pinia 或 Redux 管理对话树结构，支持多轮对话回溯和分支编辑。
 - 插件化渲染：建立渲染注册表，根据消息类型（text, image, file, tool_call）动态匹配渲染组件。
 - 离线能力：利用 IndexedDB 存储历史记录，实现秒开体验。
- 常见坑：**
- 状态结构设计太扁平，难以支持“重新生成”产生的对话分支。
 - 忽略了多模态文件上传的预签名和分片逻辑。

2. AI 产品的 Token 成本很高，作为前端，你有哪些手段可以帮助公司节省成本？

难度：★★★
考点：成本优化、技术方案权衡
答案要点：

- 前端可以通过减少请求频率、压缩上下文长度以及利用端侧能力来显著降低 Token 消耗。
- 上下文剪裁：在发送请求前，前端根据 Token 算法（如 tiktoken）计算长度，主动剔除无关的历史消息。
 - 缓存策略：对重复的查询结果进行本地缓存（如 Embedding 结果或常见问题的回答）。

- 任务分流：简单的逻辑（如格式校验、初步分类）改用正则表达式或端侧轻量级模型处理。
- 交互优化：引导用户提供更精准的输入，减少无效的反复迭代。

常见坑：

- 盲目截断上下文导致模型失去记忆。

3. 如何建立 AI 生成内容的质量评估体系（Evaluation）？前端能做什么？

难度：★★★

考点：AI 质量保障、数据反馈

答案要点：

质量评估需要量化指标和用户反馈闭环，前端是收集这些数据的核心入口。

- 反馈机制：设计多维度的点赞/点踩、修改建议、耗时统计等埋点。
- A/B Testing：同时调用两个模型或两组 Prompt，记录用户的偏好选择。
- 自动化评测：开发内部评测工具，支持批量运行测试用例并对比模型输出与标准答案的相似度。
- 性能监控：监控首字响应时间（TTFT）和整体生成速度。

常见坑：

- 只关注模型准确率，忽略了响应延迟对用户体验的影响。

4. 谈谈 WebGPU 对未来前端 AI 领域的影响，你有哪些关注或尝试？

难度：★★★

考点：技术视野、前瞻性

答案要点：

WebGPU 提供了直接访问 GPU 硬件的能力，将极大提升浏览器端运行大模型（LLM）和图像处理的性能。

- 端侧推理：未来更多的 AI 任务（如背景虚化、实时翻译、轻量级 LLM）将直接在用户浏览器运行，无需消耗服务器 GPU。
- 隐私保护：数据无需上传云端，在本地即可完成向量化和检索。
- 现状：目前 WebGPU 在 Chrome 等主流浏览器已落地，像 WebLLM 这样的项目已经证明了可行性。
- 挑战：模型权重的加载体积和显存占用仍是移动端的主要瓶颈。

常见坑：

- 夸大 WebGPU 的现状，忽略了兼容性和模型大小限制。

5. 在一个 AI 快速迭代的团队中，你如何平衡业务交付压力与技术架构的优雅性？

难度：★★

考点：软技能、工程管理

答案要点：

核心策略是“先跑通再优化”，通过模块化设计为未来的重构留出空间。

- 最小可行性（MVP）：优先保证核心链路（Prompt -> Model -> UI）跑通，不追求过度设计。
- 接口抽象：对 AI 服务进行抽象封装，确保未来更换模型供应商或协议（如从 OpenAI 换到 Anthropic）时改动最小。
- 技术债管理：记录快速迭代中留下的坑（如硬编码的 Prompt），在业务平稳期安排专项清理。
- 自动化：通过 CI/CD 和自动化测试减少回归压力。